AI Code Reviewer & Bug Detector

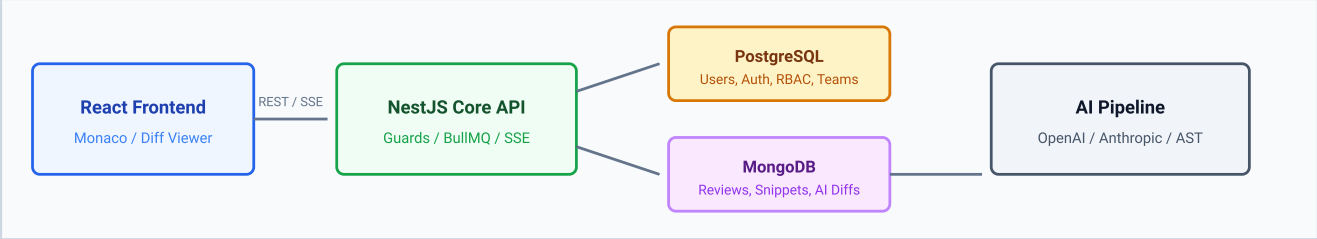
SYSTEM ARCHITECTURE & TECHNICAL BLUEPRINT

Stack: React, NestJS, PostgreSQL, MongoDB **Target:** Enterprise Full-Stack SaaS

1. Executive System Overview

This engineering blueprint specifies the construction of an automated AI code review platform. The system ingests developer code, runs multi-stage static analysis, stream-analyzes logic/security flaws using Large Language Models (LLM), and renders interactive side-by-side code diffs with line-anchored diagnostic annotations.

End-to-End Data & Execution Flow



2. System Technology Stack Matrix

Layer	Primary Technology	Core Responsibility
Frontend	React 18, Monaco Editor	Code input, syntax highlight, AST diff display, SSE client
Backend Core	NestJS, TypeScript	REST APIs, Guards, BullMQ workers, SSE streaming controllers
Relational DB	PostgreSQL + Prisma ORM	User identity, password hashes, team access control, billing
Document DB	MongoDB + Mongoose	Raw source snippets, structured LLM diagnostics, history logs
Async Queue	Redis + BullMQ	Background LLM job scheduling, token rate limiting, retries

> **Dual-Database Architecture Rationale**
> **PostgreSQL** handles strict transactional user/team relational boundaries with foreign-key guarantees. **MongoDB** stores variable-length code artifacts, unstructured AST payloads, and flexible JSON outputs returned by LLMs without requiring complex table migrations.

3. Security, Authentication & Role-Based Access

The authentication layer utilizes passport-jwt combined with GitHub OAuth2. Structured user identity lives inside PostgreSQL, mapping roles across organizations to ensure private code snippets are isolated.

NestJS Role-Based Authorization Guard

Access control is managed via custom metadata decorators and guards enforcing execution checks prior to controller invocation:

```
@Injectable() export class RolesGuard implements CanActivate { constructor(privat
e reflector: Reflector) {} canActivate(context: ExecutionContext): boolean { const roles
= this.reflector.get<string[]>('roles', context.getHandler()); if (!roles) return true;
const { user } = context.switchToHttp().getRequest(); return roles.some(role => user.rol
es?.includes(role)); } }
```

4. AI Engine & Structured Prompt Engineering

To eliminate fragile regex parsing of free-form text, the NestJS LLM service leverages API Structured Outputs (JSON Mode). This guarantees that returned payloads strictly conform to a runtime-safe JSON schema.

Enforced Diagnostic JSON Response Schema

```
{ "summary": "String: Executive assessment of code health", "severityScore": "Num
ber: 1 (Clean) to 10 (Critical Vulnerability)", "issues": [ { "lineStart": "Number", "li
neEnd": "Number", "severity": "CRITICAL | HIGH | MEDIUM | LOW", "category": "SECURITY |
BUG | PERFORMANCE | STYLE", "description": "String: Detailed explanation of defect", "su
ggestedFix": "String: Corrected code snippet" } ], "refactoredCode": "String: Complete r
efactored file content" }
```

Prompt Context Framing

System prompts clearly delimit user code using Markdown blocks. Prompts instruct the model to act as a Principal Security Architect, enforcing strict adherence to OWASP Top 10 guidelines and language-specific idiomatic standards.

5. Step-by-Step System Implementation Roadmap

Construction is broken down into three sequential development phases to maintain continuous velocity and testability:

- **Phase 1: Foundation & Identity (Weeks 1–2)**: Initialize NestJS and React repos. Setup Prisma/PostgreSQL and Mongoose/MongoDB connections. Implement JWT auth and user registration endpoints.
- **Phase 2: Monaco & LLM Integration (Weeks 3–4)**: Integrate Monaco Editor in React. Build NestJS LLM Service supporting structured JSON responses. Connect side-by-side Monaco DiffViewer.
- **Phase 3: Real-Time Features & Queues (Weeks 5–6)**: Implement SSE streaming for live reviews. Enqueue heavy jobs in BullMQ/Redis. Add review history dashboard and share link guards.

6. High-Throughput & Real-Time Infrastructure

LLM requests are computationally expensive and can take several seconds. To ensure sub-second UI responsiveness, the platform splits request handling between real-time event streaming and asynchronous queue workers.

STREAMING **Server-Sent Events**

NestJS `@Sse()` streams LLM tokens to React as they are generated, eliminating static wait spinners and providing instant feedback.

QUEUE **BullMQ + Redis**

Offloads heavy multi-file reviews and GitHub Webhook triggers to background threads, protecting the main API from event-loop blockages.

UI **Monaco Diff Engine**

Computes visual character and line differences between original source and LLM refactored code using Myers diff algorithm.

CACHE **Redis Token Cache**

Hashes source code inputs with SHA-256. Identical review submissions hit Redis cache instantly without re-querying third-party LLMs.

7. Production Security Guardrails & Pre-Filters

Before forwarding user code to third-party AI endpoints, the backend executes pre-filtering checks to minimize API costs and prevent security leaks:

- **Secret Scanning**: Runs regex rules matching AWS keys, JWTs, and database URIs to block sensitive credentials from leaving the API boundary.
- **AST Linter Pre-Check**: Evaluates basic syntax errors using lightweight linter parsers (ESLint/Ruff) before calling the LLM.
- **Rate Limiting**: Uses `@nestjs/throttler` to enforce per-user IP and token bucket quotas.

8. GitHub Webhook & PR Automation Workflow

The platform exposes a `POST /webhooks/github` route. When a pull request opens, NestJS fetches the modified git diff, queues an evaluation job in BullMQ, and posts structured inline review comments directly onto the pull request via `@octokit/rest`.

9. Target Directory Structure & Priority Matrix

```
      apps/ ├── backend (NestJS) | ├── src/auth/ # OAuth2, JWT Strategy, Guards | ├── src/re
views/ # MongoDB Schema & Review Controllers | ├── src/llm/ # OpenAI / LangChain Integration
| └── src/queues/ # BullMQ Job Processors └── frontend (React) ├── src/components/ # Monaco E
ditor, Diff Viewer, Severity Badges └── src/services/ # SSE Client & Axios Interceptors
```

Module	Priority	Complexity	Impact
Monaco Editor & Diff Viewer	HIGH	Medium	Core User Experience
Structured LLM Parsing Engine	HIGH	High	Audit Quality
JWT & PostgreSQL Auth/RBAC	HIGH	Low	Data Isolation
Server-Sent Events (SSE)	MEDIUM	Medium	Latency Reduction
GitHub Webhook PR Bot	LOW	High	Enterprise Workflow